

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №3
по дисциплине
«Управление данными»

Выполнил студент гр. ИВТб-2302-06-00

_____ /Терешенко Н.А./

Проверил преподаватель

_____ /Клюкин В.Л./

Киров

2026

Цель работы

Освоить разработку серверных программных модулей в СУБД PostgreSQL: написание пользовательских функций на языке PL/pgSQL (в том числе функций, возвращающих наборы строк, массивы и использующих динамический SQL), создание составных типов данных и обработку исключительных ситуаций, а также разработку триггеров для автоматического ведения журналов изменений.

Ход работы

Задание 1. Функция сохранения пользователя

Разработана функция `save_users`, которая добавляет нового пользователя, а при существовании записи с таким же номером удостоверяющего документа (`id_card_num`) обновляет её данные (так называемый UPSERT). Функция возвращает идентификатор пользователя, участвовавшего в операции.

Исходный код

```
CREATE OR REPLACE FUNCTION save_users(fn VARCHAR(50), ln VARCHAR(50), pn VARCHAR(20),
RETURNS BIGINT
AS $$
DECLARE
    u_id BIGINT;
BEGIN
    SELECT user_id INTO u_id FROM users WHERE id_card_num = idc;
    IF u_id IS NULL THEN
        INSERT INTO users(first_name, last_name, phone_number, id_card_num)
        VALUES (fn, ln, pn, idc)
        RETURNING user_id INTO u_id;
    ELSE
        UPDATE users
        SET
            first_name = fn,
            last_name = ln,
            phone_number = pn
```

```

        WHERE user_id = u_id;
    END IF;
    RETURN u_id;
END;
$$ LANGUAGE PLPGSQL;

SELECT save_users('Ежик', 'Соник', '+123456', 'ID-1001');
SELECT save_users('Стив', 'Майнкрафт', '+1234056', 'ID-10021');
SELECT * FROM users;

```

Результат

	123 user_id	AZ first_name	AZ last_name	AZ phone_number	AZ id_card_num
1	1	Иван	Иванов	+79991112233	123456
2	2	Алексей	Петров	+79992223344	0909
3	3	Мария	Сидорова	+79993334455	345678
4	4	Елена	Смирнова	+79994445566	456789
5	5	Дмитрий	Кузнецов	+79995556677	567890
6	6	Ежик	Соник	+123456	ID-1001
7	7	Стив	Майнкрафт	+1234056	ID-10021

Задание 2. Функция удаления тарифа с обработкой исключений

Разработана функция `delete_tiers`, которая удаляет тариф по идентификатору. Если на удаляемую строку существуют внешние ссылки из других таблиц, операция `DELETE` вызывает ошибку, которая перехватывается блоком `EXCEPTION` и заменяется пользовательским сообщением.

Исходный код

```

DROP FUNCTION IF EXISTS delete_tiers(INT);

CREATE OR REPLACE FUNCTION delete_tiers(_id INT)
RETURNS VOID
AS $$
BEGIN
    DELETE FROM tiers
    WHERE tier_id = _id;
EXCEPTION

```

```

        WHEN foreign_key_violation OR integrity_constraint_violation THEN

            RAISE EXCEPTION 'Невозможно выполнить удаление, так как есть внешние ссылки';

    END;

    $$ LANGUAGE plpgsql;

SELECT delete_tiers(5);

SELECT * FROM tiers;

SELECT delete_tiers(1);

```

Результат

	123 tier_id	AZ tier_name	123 timing_days	123 price	☒ pool_included
1	1	Базовый 1 месяц	30	2,500	[]
2	2	Стандарт 3 месяца	90	6,500	[v]
3	3	Премиум 12 месяцев	365	20,000	[v]
4	4	Базовый + Бассейн	30	3,500	[v]

Error occurred during SQL script execution
 SQL Error [P0001]: ERROR: Невозможно выполнить удаление, так как есть внешние ссылки
 Where: PL/pgSQL function delete_tiers(integer) line 7 at RAISE

Задание 3. Функция, возвращающая набор строк

Разработана функция `min_val_product`, которая возвращает набор строк таблицы `product` со значением `sale_price` не меньше заданного порога. Возвращаемый тип задан через `SETOF product`.

Исходный код

```

CREATE OR REPLACE FUNCTION min_val_product(num INT)
RETURNS SETOF product
AS $$
BEGIN
    RETURN QUERY SELECT * FROM product WHERE sale_price >= num;
END;
$$ LANGUAGE PLPGSQL;

SELECT * FROM min_val_product(1000);

```

Результат

	123 product_id	123 sale_price
1	4	1,200

Задание 4. Составной тип данных и фильтрация массива

Определён составной тип `tier_info`, описывающий тариф. Функция `filter_tier_info` принимает массив значений этого типа и возвращает новый массив, в который попадают только тарифы с ценой не ниже заданного значения.

Исходный код

```
DROP TYPE IF EXISTS tier_info CASCADE;

CREATE TYPE tier_info AS (
    tier_name VARCHAR(50),
    timing_days INT,
    price NUMERIC(10, 2)
);

CREATE OR REPLACE FUNCTION filter_tier_info(
    arr tier_info[],
    num INT
)
RETURNS tier_info[]
AS $$
BEGIN
    RETURN ARRAY(
        SELECT (tier_name, timing_days, price)::tier_info
        FROM unnest(arr)
        WHERE price >= num
    );
END;
$$ LANGUAGE PLPGSQL;

SELECT filter_tier_info(
```

```

ARRAY(
    SELECT (tier_name, timing_days, price)::tier_info
    FROM tiers
),
1500
);

```

Результат

filter_tier_info		
AZ tier_name	123 timing_days	123 price
▼ [4]	▼ [4]	▼ [4]
Базовый 1 месяц	30	2,500
Стандарт 3 месяца	90	6,500
Премиум 12 месяце	365	20,000
Базовый + Бассейн	30	3,500

Задание 5. Триггер для журналирования изменений

Создана таблица `log_tiers` для хранения журнала изменений тарифов. Триггерная функция `trigger_function` фиксирует каждую вставку и обновление строки таблицы `tiers`, сохраняя старое и новое значение цены вместе с временем изменения. После этого на таблицу устанавливается триггер `tiers_log_trigger`.

Исходный код

```

DROP TABLE IF EXISTS log_tiers CASCADE;
CREATE TABLE log_tiers (
    id BIGSERIAL PRIMARY KEY,
    tier_id INT REFERENCES tiers(tier_id),
    change_datetime TIMESTAMP DEFAULT NOW(),
    old_value NUMERIC(10, 2) DEFAULT NULL,
    new_value NUMERIC(10, 2) DEFAULT NULL
);

CREATE OR REPLACE FUNCTION trigger_function()
RETURNS TRIGGER
AS $$

```

```

DECLARE
    old_val NUMERIC(10, 2);
BEGIN
    IF (TG_OP = 'UPDATE') THEN
        old_val := OLD.price;
    ELSIF (TG_OP = 'INSERT') THEN
        old_val := NULL;
    END IF;

    INSERT INTO log_tiers (tier_id, old_value, new_value)
    VALUES (NEW.tier_id, old_val, NEW.price);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS tiers_log_trigger ON tiers;
CREATE TRIGGER tiers_log_trigger
AFTER INSERT OR UPDATE ON tiers
FOR EACH ROW
EXECUTE FUNCTION trigger_function();

INSERT INTO tiers (tier_name, timing_days, price, pool_included) VALUES ('Студенческий
UPDATE tiers SET price = 1200.00 WHERE tier_id = 1;
SELECT * FROM log_tiers;

```

Результат

	123 id	123 tier_id	change_datetime	123 old_value	123 new_value
1	1	6	2026-09-19 11:34:58.138	[NULL]	1,600
2	2	1	2026-09-19 11:34:58.143	2,500	1,200

Задание 6. Динамический SQL

Разработана функция `get_value_by_id`, которая с помощью динамического SQL (EXECUTE) выполняет сформированный на лету запрос к произвольной таблице и возвращает значение заданного столбца для строки с указанным идентификатором. Динамический SQL

строится с использованием `quote_ident` для безопасной подстановки имён объектов и параметрического `USING` для значений.

Исходный код

```
CREATE OR REPLACE FUNCTION get_value_by_id(
    tableName VARCHAR,
    columnName VARCHAR,
    idColumn VARCHAR,
    idValue BIGINT
)
RETURNS TEXT
AS $$
DECLARE
    result TEXT;
BEGIN
    EXECUTE 'SELECT ' || quote_ident(columnName) ||
        ' FROM ' || quote_ident(tableName) ||
        ' WHERE ' || quote_ident(idColumn) || ' = $1'
    INTO result
    USING idValue;

    RETURN result;
END;
$$ LANGUAGE plpgsql;

SELECT get_value_by_id('users', 'first_name', 'user_id', 1);
```

Результат

AZ get_value_by_id ▼	
1	Иван

Вывод

В ходе выполнения лабораторной работы были освоены приёмы программирования на языке PL/pgSQL в СУБД PostgreSQL. Созданы пользовательские функции с различными типами возвращаемых значений: одиночное значение, набор строк, массив составного типа, а также функция, использующая динамический SQL. Рассмотрена обработка исключительных ситуаций при нарушении внешних ключей. Разработан триггер, автоматически ведущий журнал изменений таблицы. Все функции проверены практическими запросами.